Temporal Prolog

Normative Language Specification and 1986 Paper Errata

Temporal Prolog project

Version 1.0 — 21 July 2026

Abstract

This document specifies Temporal Prolog as implemented by this project. Its source is Takashi Sakuragawa's 1986 paper, but this specification fills in the paper's intentionally informal concrete syntax, makes every temporal operator denotationally explicit, and corrects inconsistencies identified in the published transformation. The specification separates the mathematical language from the finite, range-restricted executable profile. Both the Haskell and Rust implementations are required to accept the same profile, normalize it to the same semantics (up to fresh-name alpha equivalence), and produce the same set of minimal worlds.

Contents

1 Status and conformance

The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are normative. A conforming implementation has two layers.

Core semantics. The trace and minimal-model semantics in Sections 3–6. This layer is the meaning of a program, including programs with recursion through negation.

Executable profile. The decidable operational subset in Section 7. Implementations MUST reject inputs outside the profile rather than silently assign them a different meaning.

The words "the paper" below refer to Sakuragawa, *Temporal Prolog*, RIMS Kōkyūroku 586 (1986), pp. 305–329.

2 Lexical and concrete syntax

2.1 Lexical classes

A variable begins with an uppercase ASCII letter and continues with ASCII letters, digits, or underscores. A name begins with a lowercase ASCII letter or underscore and has the same continuation characters. Decimal integer literals may have a leading minus sign and denote arbitrary-precision mathematical integers. Their spelling is canonicalized by numeric value, so `007` denotes the same term as `7` and `-0` denotes `0`. A comment begins with `%` and extends to the end of the line. Whitespace and comments separate tokens. Identifiers containing non-ASCII characters MUST be rejected; Unicode is accepted only for the operator aliases listed below.

Keyword reservation is contextual because predicates and constructors occupy separate namespaces. The formula-control words `since`, `after`, `for`, `until`, `atnext`, `always`, `eventually`, and `next` MUST NOT be used as predicate or pattern- function names, but MAY be used as constructor names in term position. Names whose special meaning belongs to only one namespace are not globally reserved. In particular, `div` and `mod` MAY be predicate names even though their arity-two term forms are arithmetic, while external predicates such as `true` and `is` MAY also occur as constructor names inside terms. The call spellings `true()` and `false()` are equivalent to their bare forms, and `is(L,R)` is equivalent to `L is R`.

The namespaces below are disjoint in the abstract language:

$$V, \quad SF, \quad PF_i, \quad PF_e, \quad P_i, \quad P_e.$$

They denote variables, constructor (Skolem) functions, internal and external pattern functions, and internal and external predicates. Each non-variable symbol has a fixed arity. Constants are arity-zero constructor functions. The external predicates include $=$, `is`/2, the four integer comparisons, `true`, `false`, and `at`/1.

The concrete implementation infers internal pattern-function names from reduction heads. Other callable names are predicates; constructor names are identified by term position. Predicate and constructor signatures are tracked separately, so the same spelling MAY have one predicate arity and one constructor arity. Within either namespace, a program MUST use a symbol at one fixed arity.

Every internal pattern-function name has one input arity k across all of its reductions. Its term form has arity k, while its relational atom form has arity $k + 1$, with the final argument denoting the result. Once a reduction declares a pattern-function spelling, other uses of that spelling MUST obey these two signatures. External predicates have the fixed signatures listed above and MAY occur only as conditions; they MUST NOT occur in rule results or be inserted into a world as external input. The arithmetic symbols $+$, $-$, $*$, `div`, and `mod` have fixed arity two in term position. An implementation MUST validate these

constraints before normalization and reject the whole source program on a violation. The same signatures remain fixed across runtime queries, pending assertions, and every previously computed world; runtime input MUST NOT introduce a conflicting arity.

The concrete spelling `prefix_auxN` is not reserved: a source predicate with that spelling remains a source predicate and MUST remain observable. Implementations therefore track generated predicates by normalization provenance rather than classifying names lexically.

2.2 Grammar

The following EBNF is normative for the portable ASCII syntax. Unicode aliases are listed in Table 1.

```
program      ::= item*
item         ::= rule "." | reduction "."
rule         ::= result | condition "=>" result
reduction    ::= call "->" term

result       ::= result-and
                 [("until" | "atnext") condition]
result-and   ::= result-unary ("/\\" result-unary)*
result-unary ::= "always" result-unary
               | "next" result-unary
               | result-atom
result-atom  ::= atom | reduction | "(" result ")"

condition       ::= condition-and
                    [("since" | "after") condition-and
                     | "for" positive-integer]
condition-and   ::= condition-unary ("/\\" condition-unary)*
condition-unary ::= ("~" | "@" | "#" | "?") condition-unary
                  | "eventually" condition-unary
                  | condition-atom
condition-atom  ::= atom | "(" condition ")"

atom         ::= call | name
               | term ("=" | "!=" | "\\=" | "is"
                       | ">" | "<" | ">=" | "<=") term
call         ::= name "(" [term ("," term)*] ")"

term         ::= variable | integer | name | call
               | "[" [term ("," term)* ["|" term]] "]"
               | "@" term
               | term ("+" | "-" | "*" | "div" | "mod") term
               | "(" term ")"
```

Unary operators bind most tightly. Condition conjunction binds more tightly than `since`, `after`, and `for`. Those three binary past-time operators are non-associative; chained uses require parentheses. Within a result, `always` and `next` bind most tightly, result conjunction binds next, and `until` and `atnext` bind least tightly. The latter two operators are non-associative: nested uses require parentheses. Thus `p /\ q until stop` means `(p /\ q) until stop`. Implication binds least tightly overall.

ASCII	Unicode	Meaning
`=>`	$\Rightarrow$	implication
`/\`	$\wedge$	conjunction
`->`	$\rightarrow$	reduction
`@`	$\bullet$	previous time
`~`	$\neg$	negation
`#`	$\blacksquare$	continuously in the past
`?`	$\blacklozenge$	once in the past
`eventually`	$\Diamond$	once in the past
`always`	$\Box$	henceforth
`next`	$\bigcirc$	next time

Table 1: Portable ASCII spellings and accepted Unicode aliases.

Both `U+25CF` BLACK CIRCLE and `U+2022` BULLET are accepted aliases for `@`. `U+25C6` BLACK DIAMOND is the Unicode spelling of `?`, while `U+25C7` WHITE DIAMOND is the Unicode spelling of `eventually`. Implementations MUST preserve those operator families in the parsed source AST, even though `?` and `eventually` normalize to the same past-existential semantics.

2.3 Well-formed terms and formulas

Terms are generated by

$$t ::= X \mid f(t_1, \ldots, t_k) \mid \bullet t.$$

The paper restricts $\bullet t$ to a term containing at least one occurrence of a pattern function. The executable profile enforces that restriction. In particular, `@X` is not a portable way to refer to a variable's old binding. Previous-time pattern-function values are defined in Section 5.3.

An atom is $p(t_1, \ldots, t_k)$. A condition is an atom closed under negation, previous time, past operators, and conjunction. A result is an internal atom or internal reduction closed under implication, conjunction, and future operators. Free variables in each top-level item are universally quantified.

3 Traces and temporal truth

A trace is an infinite sequence $M = \langle w_0, w_1, \ldots \rangle$. Each w_n is a set of ground atoms. Constructor interpretations and the Herbrand domain are time invariant; predicate truth may vary with time. We write $M, n, \rho \models c$ for condition c at time n under grounding environment ρ.

Atomic truth, conjunction, and negation are classical at a fixed world:

$$\begin{aligned} M, n, \rho &\models a && \iff \rho(a) \in w_n, \\ M, n, \rho &\models \neg c && \iff M, n, \rho \not\models c, \\ M, n, \rho &\models c \wedge d && \iff M, n, \rho \models c \text{ and } M, n, \rho \models d. \end{aligned}$$

Previous time is false before the trace begins, irrespective of its operand:

$$M, n, \rho \models \bullet c \iff n > 0 \text{ and } M, n-1, \rho \models c.$$

Consequently $\bullet \neg p$ is false at w_0, while $\neg \bullet p$ is true there. An implementation MUST NOT commute negation through $\bullet$ at the initial world.

The derived past operators have the following meanings:

$$
\begin{aligned}
M, n, \rho \models \blacksquare c &\iff \forall i \ (0 \le i \le n \Rightarrow M, i, \rho \models c), \\
M, n, \rho \models \blacklozenge c &\iff \exists i \ (0 \le i \le n \wedge M, i, \rho \models c), \\
M, n, \rho \models c \ \mathsf{since}\ d &\iff \exists j \le n \,. M, j, \rho \models d \wedge \forall k \, (j \le k \le n \Rightarrow M, k, \rho \models c), \\
M, n, \rho \models c \ \mathsf{after}\ d &\iff \exists i, j \,. 0 \le i < j \le n \wedge M, i, \rho \models d \wedge M, j, \rho \models c, \\
M, n, \rho \models c \ \mathsf{for}\ k &\iff k > 0 \wedge n \ge k - 1 \wedge \forall i < k \,. M, n - i, \rho \models c.
\end{aligned}
$$

The `after` relation is strict and, once witnessed, remains true. This is the paper's prose definition; it corrects the inconsistent printed Step 2(4), as explained in Section 9.

The built-in $\mathsf{at}(k)$ is true exactly at w_k. true is always true and false always false. Equality is syntactic equality in the Herbrand universe.

4 Result obligations

A top-level item is required to hold at every time. We define $M, n, \rho \models_R r$ as follows.

An atomic result a requires $M, n, \rho \models a$. A reduction result is defined relationally in Section 5.3. Conjunction requires both results. Implication has the usual meaning:

$$
M, n, \rho \models_R c \Rightarrow r \iff M, n, \rho \not\models c \text{ or } M, n, \rho \models_R r.
$$

Future result operators create obligations from the evaluation point:

$$
\begin{aligned}
M, n, \rho \models_R \Box r &\iff \forall k \ge n \,. M, k, \rho \models_R r, \\
M, n, \rho \models_R \bigcirc r &\iff M, n + 1, \rho \models_R r.
\end{aligned}
$$

For $r \ \mathsf{until}\ c$, let j be the least $k \ge n$ satisfying c, if one exists. The result r is required at every k with $n \le k < j$; if no such j exists, r is required forever from n. For $r \ \mathsf{atnext}\ c$, r is required at that least j and no obligation is imposed before it. If c never occurs, `atnext` imposes no result obligation.

A program P is satisfied by M iff every universally grounded top-level item holds at every $n \ge 0$.

5 Normalization

Normalization produces clauses

$$
\bullet^{m_1}(\neg) c_1 \wedge \cdots \wedge \bullet^{m_k}(\neg) c_k \Rightarrow d,
$$

where each c_i is atomic, each $m_i \ge 0$, and d is an internal atom. Fresh predicates and variables MUST be absent from the source program. Rules below are equivalences under the minimal-model semantics; fresh names are local to a normalization run.

5.1 Step 1: future results

Result conjunction distributes into separate clauses. Bare future formulas simplify as

$$
\Box q \rightsquigarrow q, \qquad q \ \mathsf{until}\ a \rightsquigarrow \neg a \Rightarrow q, \qquad q \ \mathsf{atnext}\ a \rightsquigarrow a \Rightarrow q.
$$

For a conditional always result, with fresh $p(\vec{X})$ and $\vec{X} = \mathrm{fv}(q)$:

$$\begin{aligned} a \Rightarrow \Box q \quad \rightsquigarrow \quad & a \Rightarrow p(\vec{X}), \\ & \bullet\, p(\vec{X}) \Rightarrow p(\vec{X}), \\ & p(\vec{X}) \Rightarrow q. \end{aligned}$$

For conditional until, let $\vec{X} = \mathrm{fv}(q, b)$:

$$\begin{aligned} a \Rightarrow q \;\mathsf{until}\; b \quad \rightsquigarrow \quad & a \Rightarrow p(\vec{X}), \\ & \bullet\, p(\vec{X}) \wedge \bullet\neg b \Rightarrow p(\vec{X}), \\ & p(\vec{X}) \wedge \neg b \Rightarrow q. \end{aligned}$$

Conditional `atnext` uses the same first two clauses and replaces the third with $p(\vec{X}) \wedge b \Rightarrow q$. The project extension $a \Rightarrow \bigcirc q$ becomes $a \Rightarrow p(\mathrm{fv}(q))$ and $\bullet p(\mathrm{fv}(q)) \Rightarrow q$.

5.2 Step 2: past conditions

For each occurrence below, replace the occurrence by p in its surrounding clause. Predicate arguments are exactly the displayed free-variable set, not the variables of the surrounding clause.

Occurrence	Defining clauses
$\blacksquare a$	$a \wedge \mathsf{at}(0) \Rightarrow p(\mathrm{fv}(a))$; $\bullet p(\mathrm{fv}(a)) \wedge a \Rightarrow p(\mathrm{fv}(a))$
$\blacklozenge a$	$a \Rightarrow p(\mathrm{fv}(a))$; $\bullet p(\mathrm{fv}(a)) \Rightarrow p(\mathrm{fv}(a))$
a since b	$b \wedge a \Rightarrow p(\mathrm{fv}(a, b))$; $\bullet p(\mathrm{fv}(a, b)) \wedge a \Rightarrow p(\mathrm{fv}(a, b))$
a after b	$b \Rightarrow s(\mathrm{fv}(b))$; $\bullet s(\mathrm{fv}(b)) \Rightarrow s(\mathrm{fv}(b))$; $\bullet s(\mathrm{fv}(b)) \wedge a \Rightarrow p(\mathrm{fv}(a, b))$; $\bullet p(\mathrm{fv}(a, b)) \Rightarrow p(\mathrm{fv}(a, b))$
a for k	replace with $a \wedge \bullet a \wedge \cdots \wedge \bullet^{k-1} a$, where $k > 0$

The auxiliary s in `after` records that the right operand occurred; p records a later left-operand witness. Using $\bullet s$ makes the order strict. Both predicates retain exactly the variables necessary to relate their witnesses.

5.3 Step 3: pattern functions

An internal reduction

$$c \Rightarrow f(t_1, \ldots, t_k) \to t_0$$

becomes the relational clause

$$c \Rightarrow f(t_1, \ldots, t_k, t_0).$$

The new predicate has arity $k + 1$. Multiple reductions denote a relation; the language does not silently impose functional uniqueness.

For the outermost, rightmost pattern-function occurrence $f(t_1, \ldots, t_k)$ inside a predicate term, introduce fresh X, replace the occurrence by X, and conjoin $f(t_1, \ldots, t_k, X)$. If the occurrence lies under m term-level previous operators, conjoin $\bullet^m f(t_1, \ldots, t_k, X)$ while leaving the enclosing predicate at its original condition depth. Repeat until no pattern function occurs in a term, then erase every remaining term-level $\bullet$ wrapper.

For example,

$$\mathsf{present}(\bullet\mathsf{lookup}(k)) \rightsquigarrow \mathsf{present}(X) \wedge \bullet\mathsf{lookup}(k, X).$$

Moving `present` into the previous world would be non-conformant.

5.4 Steps 4 and 5: negation and previous time

If $\neg a$ occurs and a is not atomic, introduce $a \Rightarrow p(\mathrm{fv}(a))$ and replace the occurrence by $\neg p(\mathrm{fv}(a))$. This rule is essential for $\neg \bullet a$: directly rewriting it as $\bullet\neg a$ is invalid at w_0.

Finally distribute $\bullet$ over conjunction. A normal condition has exactly the shape $\bullet^m a$ or $\bullet^m \neg a$.

6 Models and nondeterminism

Past worlds and external atoms at the current world are fixed inputs. For a normal program, remove all previous-time body atoms when constructing the current dependency graph. There is an edge $p \to q$ when a current-world atom with predicate q occurs in the body of a clause headed by predicate p; retain whether the occurrence is positive or negative.

Collapse strongly connected components (SCCs) and order them so every dependency SCC precedes its consumers. Within a current world, compare two models lexicographically by the sets of atoms in this SCC order: at the first component where they differ, a model is smaller only when its component set is a proper subset of the other's. Incomparable component sets remain incomparable. A model is minimal when no model is smaller by this order.

If no SCC contains a negative edge, the program satisfies the paper's condition 1 and has a unique least model. It is computed SCC-by-SCC by least fixed points. Previous-time conditions do not create current-world dependencies.

If a negative edge lies inside an SCC, a conforming implementation MUST NOT pretend that the program is stratified. Its semantics is the set of minimal models. An execution MAY choose any member, and an API intended for testing SHOULD expose all members. For example, the clauses printed in paper Section 4.7 (made range restricted here by an explicit domain condition) are:

```
assign(X) /\ ~assigned_to_another(X) => assigned_to(X).
assigned_to(X) /\ assign(Y) /\ X != Y => assigned_to_another(Y).
```

With simultaneous requests from 1 and 2, the paper claims two minimal worlds, one assigning each requester. Under its stated model order there is also a third: the "assigned to another" blocker is true for both requesters and neither requester is assigned. Neither blocker can be removed without forcing an incomparable assignment, so this world is minimal. Implementations MUST expose all three for the published program. The following corrected encoding has exactly the intended two minimal worlds:

```
assign(X) /\ assign(Y) /\ X != Y /\ ~assigned_to(Y)
  => assigned_to(X).
```

The mathematical language calls a program legal when it has at least one model for every interpretation of its external predicates. This property is not decidable for the unrestricted Herbrand universe.

7 Executable profile

The Haskell and Rust implementations execute the following common profile.

1. Assertions supplied for a step are ground and form part of the fixed external input for that step. They MUST preserve predicate and constructor signatures established by the normalized program and all earlier or pending assertions. A newly introduced input symbol establishes its signature for the remainder of the run.
2. Assertions MUST NOT use an external-predicate name, an internal pattern-function relation, or a predicate introduced by normalization. Runtime atoms MUST NOT contain a term-level `@` wrapper or a malformed arithmetic operator.
3. Query atoms obey the same signature rules and MUST NOT address normalization-generated predicates. Internal pattern-function relations MAY be queried at their relational arity.
4. Every variable used by a negated condition or a forward-chained head is grounded by positive body conditions, $\mathsf{at}(X)$, unification with a ground term, or a ground arithmetic evaluation before it is observed.
5. Candidate generation for a world reaches a finite fixed point within the configured fuel limit. A program that generates an unbounded Herbrand base is rejected for that run.
6. The general evaluator's candidate base is closed under relaxed positive derivation and under every grounded current-world negative atom whose predicate is internal and whose remaining body is satisfied in that relaxed closure. These steps repeat to a fixed point. This second clause is necessary because the paper uses classical minimal models: an unsupported atom may occur solely to block a negative antecedent.
7. General negative SCCs are executed only when the finite candidate base is no larger than the configured enumeration limit. Exceeding the limit is a resource error, not a semantic result.
8. Internal pattern-function relations are solved by alpha-renamed backward chaining. Reaching its recursion limit is reported as a resource error in conformance mode; it MUST NOT be reported as logical failure.

For condition-1 programs, the optimized stratified evaluator and the general minimal-model evaluator MUST agree. Differential tests compare both engines and compare the Haskell and Rust implementations on canonicalized worlds.

8 External operations and project extensions

The portable executable profile defines arbitrary-precision integer arithmetic terms $+$, $-$, $*$, `div`, and `mod`. Multiplication, `div`, and `mod` share one left-associative precedence level above addition and subtraction. The prefix forms `div(a,b)` and `mod(a,b)` are equivalent functional spellings for the corresponding arithmetic expressions.

For integers a and nonzero b, $a \operatorname{div} b$ is the quotient $q = \lfloor a/b \rfloor$, and $a \bmod b$ is the unique remainder $r = a - bq$. Thus r is zero or has the sign of b, and $|r| < |b|$. The predicate X `is` E evaluates a ground expression E and unifies X with its integer result. Comparisons require ground integer operands. Division by zero and nonground or noninteger arithmetic fail without binding and MUST NOT fall through to stored-predicate lookup. `!=` and `\=` abbreviate negated Herbrand equality. The remaining external predicates are Herbrand equality $=$, the four integer comparisons, $\mathsf{at}/1$ for the current world number, and the constants `true` and `false`. These operations are determined by the evaluator rather than world membership, so an implementation MUST reject them in scheduled assertions and model-checking input choices. A public query API MUST evaluate these predicates by the same rules used for program conditions and return their substitutions; it MUST NOT treat a well-formed successful

external query as an absent stored fact.

The project accepts `eventually c` as a spelling of $\blacklozenge c$ and `next r` as the next-result operator. These are conservative extensions and are tested by both implementations.

9 Published-paper errata and clarifications

Location	Published text or ambiguity	Normative resolution
Section 3, `after`; Step 2(4)	The prose requires a left-operand occurrence after a right-operand occurrence, but the printed recurrence merely remembers the more recent operand and is true after the left operand even if the right never occurred.	Use the strict two-latch construction in Section 5. A same-world pair is not "after," and a witnessed pair remains witnessed.
Section 3, `for`	The metavariable is first called non-negative, then explicitly positive.	The operand MUST be positive. Zero is rejected.
Step 1(2) and Step 2 auxiliary arguments	Several displayed subscripts alternate k and l, and an implementation might capture variables from the surrounding clause.	Use exactly the free-variable sets printed by the defining operator: $\mathrm{fv}(q)$, $\mathrm{fv}(q, b)$, $\mathrm{fv}(a)$, or $\mathrm{fv}(a, b)$.
Step 3 term-level previous	The paper says to erase term-level bullets after expansion but does not warn that lifting the enclosing predicate changes its time.	Only the generated pattern-function condition receives the occurrence's term depth; the enclosing atom keeps its condition depth.
Steps 4–5 at w_0	The normal form places negation inside previous operators, which can tempt an implementation to rewrite $\neg \bullet a$ as $\bullet \neg a$.	Step 4 MUST introduce an auxiliary before distribution. The two formulas differ at w_0.
Section 5.3	Condition 1 is a sufficient least-model condition, not the definition of all legal programs; Section 4.7 intentionally uses a negative SCC.	Expose minimal-model nondeterminism for finite executable cases. Rejecting every negative SCC is a supported fast-path limitation only when clearly reported, not a conformant semantic answer.
Section 4.7	The nondeterministic assignment program is claimed to have two choices, but its two unsupported `assigned_to_another` atoms form a third minimal model in which nobody is assigned.	Retain all three models for the published clauses, as required by the formal minimal-model semantics. Use the corrected direct-choice clause in Section 6 when exactly two assignment outcomes are intended.

Location	Published text or ambiguity	Normative resolution
Concrete syntax and external domains	The paper intentionally gives no machine grammar or domain-declaration syntax.	Use Section 2's grammar and Section 7's ground assertion and range-restriction rules.

10 Conformance requirements

A release is conformant only when all of the following are true.

1. Both implementations parse every shared positive corpus case and reject every shared negative case. Rejections include inconsistent symbol arities, non-ASCII identifiers, malformed pattern-function relational forms, malformed external operation signatures, and external predicates used as results or asserted world facts. Positive cases exercise contextual keyword reservation across predicate and constructor namespaces. Runtime rejection cases cover signature changes across source, pending input, and history; internal-name injection; term-level previous; and malformed arithmetic. Public-query tests cover every external predicate family and its variable bindings.
2. Normalization invariants hold: no surface temporal operator or term-level previous wrapper remains, every body condition is normal, and fresh identifiers do not collide with source identifiers. Generated-name provenance is exact: source identifiers resembling the fresh-name spelling remain observable, while only actual auxiliaries are hidden by default.
3. Canonical sets of worlds agree for every shared scenario, including initial-world negation, corrected `after`, recursive pattern functions, arbitrary-precision signed arithmetic, unsupported classical blockers, and Section 4.7 nondeterminism.
4. The stratified and general evaluators agree wherever condition 1 holds.
5. Benchmarks report workload parameters, iterations, elapsed time, and a semantic digest. A timing result without a checked digest is invalid.

References

[1] Takashi Sakuragawa. Temporal Prolog. *RIMS Kōkyūroku*, 586:305–329, 1986.

[2] Keith L. Clark. Negation as failure. In *Logic and Data Bases*, pages 293–322, 1977.